Technical Documentation

OOP Project "FixItNow"

Tudorache Robert-Cristian
Group 322AA

January 11, 2026

1 Application Architecture

The application is built on a modular architecture, strictly adhering to Object-Oriented Programming (OOP) principles. The class structure is designed to separate business logic (`Service`) from data entities (`Angajati`, `Electrocasnice`).

Class Hierarchy

1. **Service Class (Singleton):** It represents the "brain" of the application. It manages the employee and request lists. Being a Singleton, it ensures a single globally accessible instance, thus preventing data conflicts.
2. **Angajat (Employee) Hierarchy:**
   - The abstract base class *Angajat* defines common attributes (name, CNP, hiring date) and the pure virtual method `calculeazaSalariu()`.
   - The derived classes *Tehnician*, *Receptioner*, and *Supervizor* implement specific salary calculation logic (coefficients, bonuses). *Tehnician* additionally includes a list of competencies and an availability status.
3. **Electrocasnic (Appliance) Hierarchy:**
   - The abstract class *Electrocasnic* defines the generic appliance.
   - The derived classes *Frigider*, *MasinaSpalat*, and *Televizor* add specific attributes (freezer, capacity, screen size) and override the `afisare()` method.
4. **ElectrocasnicFactory:** Implements the Factory Method design pattern for centralized object creation, decoupling instantiation logic from the rest of the application code.
5. **Cerere (Request):** Links an *Electrocasnic* object to a specific status (`InAsteptare`, `Asignata`, `Finalizata`) and a repair complexity level.

Conceptual Relationship Diagram

- Service "has" (aggregation) → lists of *Angajat* and *Cerere*.
- Cerere "has" an *Electrocasnic*.
- Angajat (inherits) → *Tehnician*, *Receptioner*, *Supervizor*.

2 Mode of Operation

The application flow begins in `main.cpp`, which initializes the `Service` Singleton and loads data from the CSV files.

Main Processes

1. **Data Import:** Robust CSV parsing functions are utilized. Lines are split, and data is properly converted. Formatting errors (e.g., letters instead of numbers) are caught using `try-catch` blocks, ensuring program continuity without crashing.
2. **Control Panel (Main Loop):** The user interacts via a console menu divided into three sections: HR (Employees), Operations (Requests), and Reporting.
3. **Load Balancing Mechanism:** During the task assignment option, the Service scans the pending requests and searches for the compatible technician with the lowest number of active tasks, ensuring a fair distribution of the workload.
4. **Time Simulation:** The user inputs a simulated number of minutes. The system subtracts this time from the remaining duration of active repairs. When the duration reaches 0, the technician becomes available again and receives the financial bonus.

3 Technical Elements and Modern C++

The project utilizes modern C++ features (C++17 Standard) and STL libraries:

- **Design Patterns:**
  - *Singleton:* Used in the `Service` class for centralized data access.
  - *Factory:* The `ElectrocasnicFactory` class simplifies object creation based on a string input (e.g., "Frigider"), eliminating repetitive conditional logic within `main`.
- **STL (Standard Template Library):**
  - `std::vector`: For dynamic management of employee and request lists.
  - `std::string` and `std::stringstream`: For text manipulation and robust CSV parsing.
  - `std::sort` with *Lambda Expressions*: Utilized in generating reports (e.g., sorting employees by salary) for concise and highly readable code.
- **Memory Management:** Use of destructors for safely freeing dynamically allocated memory (chained `delete` operations) upon application shutdown.
- **Input Validation:** Extensive use of `cin.fail()`, `cin.clear()`, and `cin.ignore()` to prevent application blocks when encountering invalid keyboard input.

4 Test Description

Testing was conducted using external files (`tests/angajati.csv`, `tests/cereri.csv`) to simulate real-world scenarios ("Black Box Testing").

Covered Cases

- **CNP Validation:** The official Romanian ID validation algorithm was implemented. Tests included CNPs with incorrect lengths, non-numeric characters, and invalid control digits. All instances were correctly identified and handled.
- **Import Robustness:** CSV files contained intentionally altered lines (e.g., prices written as text, invalid years). The application reported the specific error (e.g., `stod failed`) and successfully continued loading the remaining valid data.
- **Operating Conditions:** A critical business condition was verified: the application denies access to the main service menu if the team is incomplete (minimum requirements: 3 technicians, 1 receptionist, 1 supervisor).